Dual-Sided Predictive Demand Forecasting & Real-Time B2B Inventory Management System

A comprehensive 100% free toolstack blueprint for synchronized supply chain intelligence.

Retailer Marketplace | Supplier Dashboard | AI Forecasting

> *This document outlines the foundation and detailed planning for a dual-sided, real-time B2B inventory tracking and predictive forecasting platform.*

PROJECT FOUNDATION

1. The Problem Statement

Small retail shop owners constantly face inventory mismanagement due to a lack of real-time visibility into upstream supplier stock. This information asymmetry leads to two major financial drains: excessive over-ordering that ties up working capital, and frustrating stockouts that result in lost sales. Concurrently, wholesale suppliers struggle to anticipate

aggregate retail demand, leading to inefficient warehouse procurement and order rejections.

2. The Proposed Solution

A dual-sided, real-time B2B inventory tracking and predictive forecasting platform. This system bridges the gap between wholesalers and retailers by providing a synchronized marketplace where shop owners can view live inventory levels. Furthermore, it integrates Machine Learning models to transform the supply chain from reactive to proactive, predicting both B2C (customer) demand for the retailer and B2B (aggregate) procurement needs for the supplier.

3. Core Building Modules

Identity & Access Management (IAM)

Secure authentication system directing users to either the Supplier Portal or Retailer Marketplace based on their role.

Supplier Inventory Engine

A digital warehouse management dashboard allowing wholesalers to manage catalogs, track incoming orders, and bulk-upload stock data via CSV to eliminate manual entry friction.

Real-Time Marketplace & Sync Engine

The retailer-facing e-commerce interface where shop owners can browse catalogs. Powered by WebSockets, it dynamically updates stock levels (In Stock, Low Stock, Out of Stock) instantly without page refreshes.

Retailer AI Forecasting (B2C)

An analytics module that ingests a shop's historical sales data using time-series forecasting to generate a "Smart Restock" cart, predicting exactly what items will run out and when.

Supplier AI Procurement (B2B)

A macro-level prediction model that aggregates incoming order data across all connected retailers to help the wholesaler optimize their own factory procurement and avoid stockouts.

4. User Story Map

For the Retailer

Q: *"As a shop owner, I want to see which items are running out **before** they do, so I can order them in time."*

Q: *"As a shop owner, I want to see if my supplier has stock **live**, so I don't place orders that will be rejected."*

Q: *"As a shop owner, I want to login securely with my email without remembering a complex password."*

For the Supplier

Q: *"As a wholesaler, I want to upload my entire inventory via CSV so I don't have to enter 500 items manually."*

Q: *"As a wholesaler, I want to see the total demand across all my shops so I can prepare my factory for next week."*

II TECHNOLOGY STACK & INFRASTRUCTURE

5. 100% Free Toolstack & API Ecosystem

Every tool selected for this project has a robust free tier or is open-source.

LAYER	RECOMMENDED TOOL	FREE TIER/USAGE
Frontend	React.js + Vite	Open Source
Routing	React Router	Open Source
Styling	Tailwind CSS	Open Source
Auth & DB	**Supabase**	500MB DB + 50k Users
Real-time	Supabase Realtime	Included (WebSockets)
OTP Service	Supabase Auth (Email OTP)	Unlimited Free Email OTP
Mail Delivery	**Resend**	3,000 Emails/Month

LAYER	RECOMMENDED TOOL	FREE TIER/USAGE
Core Backend	Node.js + Express	Open Source
AI Backend	FastAPI (Python)	Open Source
ML Models	Scikit-learn, Prophet	Open Source
Data Viz	Recharts	Open Source
Hosting (FE)	Vercel	Unlimited Personal
Hosting (BE)	Render	Shared Free Instance

6. Project Folder Structure

```
/
├── frontend/               # React (Vite)
│   ├── src/
│   │   ├── components/     # Reusable UI (Buttons, Cards,
Modals)
│   │   ├── hooks/          # Supabase & ML fetching hooks
│   │   ├── pages/          # SupplierDashboard,
RetailerMarketplace, Auth
│   │   ├── store/          # Zustand state (auth, cart,
inventory)
│   │   └── utils/          # CSV parsers, Date formatters
├── backend-node/           # Core Business Logic (Express)
│   ├── src/
│   │   ├── controllers/    # Order logic, Bulk upload handling
│   │   ├── middleware/     # Auth & Role validation
│   │   └── services/       # Supabase Admin client
integrations
```

```
├── backend-python/          # ML Microservice (FastAPI)
│   ├── main.py              # API routes
│   ├── models/              # Prophet & Scikit-learn wrappers
│   └── data/                # Sample datasets for training
└── supabase/                # DB Migrations & RLS Policies
```

7. $0 Budget Analysis

Every component of this plan is strictly free for the MVP/Scale phase:

✓ Database: $0 (Supabase Free Tier)

✓ Authentication: $0 (Supabase Auth)

✓ Hosting: $0 (Vercel & Render)

TOTAL MONTHLY COST

$0.00

8. Deployment Pre-flight Checklist

1. **Supabase:** Ensure RLS is enabled on all tables; otherwise, data will be public.
2. **Vercel/Render:** Set `NODE_ENV=production` and add `SUPABASE_KEY` as secret environment variables.
3. **Python:** Ensure `requirements.txt` is updated for Render's build process to install `prophet` and `pandas`.

III TECHNICAL ARCHITECTURE & SECURITY

9. System Architecture & Routing

Multi-page React application using **React Router** for distinct user experiences.

Public Routes

- / landing
- /auth (Dual Login)

Supplier Routes

- /dashboard/supplier
- /inventory/bulk-upload
- /procurement-analytics

Retailer Routes

- /marketplace
- /smart-cart
- /demand-analytics

10. High-Level Data Flows

A The AI Forecasting Pipeline (B2C)

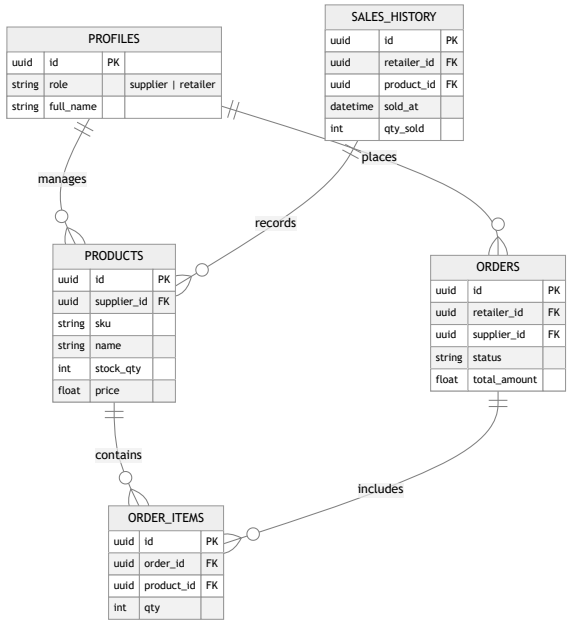
1. **Collection:** Retailer records sales via `sales_history`.

2. **Trigger:** Retailer opens "Smart Restock" page.
3. **Extraction:** Node.js backend queries last 90 days of history.
4. **Processing:** Node.js sends data to **Python FastAPI**.
5. **Inference:** Python uses `Prophet` to calculate demand.
6. **Return:** Returns JSON with `suggested_restock`.
7. **Action:** Frontend populates Smart Cart automatically.

B Real-time Inventory Sync

1. **Event:** Retailer A confirms order for 10 units.
2. **Server:** Node.js executes logic: `stock_qty = stock_qty - 10`.
3. **Broadcast:** Supabase Realtime emits `db_change` event.
4. **Frontend:** All connected clients receive WebSocket update.
5. **UI:** Badge turns yellow/red with an instant CSS pulse effect.

11. Database Schema & ERD



12. Supabase RLS Policies (Security)

`Profile` **profiles_owner:** auth.uid() == id

`Catalog` **products_supplier:** role == 'supplier'

`Market` **products_retailer:** role == 'retailer' (Read)

`Orders` **orders_access:** retailer_id == auth.uid()

13. Compliance & Data Privacy Basics

- **Encrypted Storage:** Supabase handles data encryption at rest.
- **Anonymized AI:** ML service receives only `sku` and `qty`, never personal identifiers.

- **Session Security:** JWT tokens expire in 1 hour; Supabase manages Refresh tokens.

IV APPLICATION LOGIC & STATE

14. Authentication & Entry Flow (OTP)

OTP verification with role-based redirection logic.

```
1. Entry: User enters Email + Role.
2. Request: Frontend calls signInWithOtp().
3. Delivery: Supabase sends 6-digit code via Email.
4. Verification: User enters code in UI.
5. Redirection:
   - If Supplier: /dashboard/supplier
   - If Retailer: /marketplace
```

15. Frontend Architecture & State (Zustand)

Auth Store

Session state and user identity tracking.

Inventory Store

Local product cache for real-time updates.

Cart Store

Persistent checkout state for retailers.

16. UI/UX Page Mapping

Landing & Auth: Home, Dual-Role card with Indigo/Violet themes.

Retailer: Infinite scroll grid, predictive Recharts graphs, "Smart Restock" refill button.

Supplier: Live stock table, macro-view procurement advisor with AI suggestions.

17. Detailed API & Function Signatures

Node.js	**POST /api/orders:** Transactional order logic with stock locking.
Node.js	**POST /api/inventory/bulk:** CSV stream parsing and DB Upsert.
Python	**POST /ml/forecast:** Time-series Prophet inference via FastAPI.

V AI & MACHINE LEARNING STRATEGY

18. ML Strategy Rationale

B2C Forecasting

Prophet (Meta): Robust against seasonality and missing retail data points.

B2B Procurement

XGBoost/RF: Pattern recognition across aggregate market demand metrics.

19. Mock Dataset Strategy

Python-driven synthetic data generation for demonstration readiness.

Dataset	Columns	Trend Pattern
Sales History	date, sku, qty, flag	Linear growth + weekend spikes
Inventory Logs	time, sku, delta, reason	Decrement (sales) / Increment (restock)
Order Metadata	retailer_id, duration	Log-normal (avg 3-day fulfillment)

20. Mock API Response


```
[
  {
    "sku": "APPL-001",
    "current_stock": 50,
    "projected_out_date": "2024-04-28",
    "suggested_restock_qty": 150,
    "confidence_score": 0.92,
    "trend": "rising"
  }
]
```


21. Performance Optimization

- **Caching:** React Query caches AI results for 1 hour to avoid Render free tier hits.
- **Debouncing:** Bulk uploads and search inputs are throttled.
- **UI Throttling:** Pulse effects for stock deduction are throttled for peak traffic.

VI FRONTEND & DESIGN AESTHETICS

22. UI Style Guide & Aesthetics

Background	Retailer
Zinc 900	Indigo 500

Supplier	Growth
Violet 500	Emerald 500

23. React Component Hierarchy

Common

- AuthLayout & DashboardLayout
- RoleToggle (Morphing Animation)
- OTPInput (Focus Shifting)

Specific

- MarketGrid (Retail) / InventoryTable (Supplier)
- RestockList (AI-driven suggest)
- ProcurementChart (Aggregate view)

24. Form & Validation

Using **React Hook Form + Zod** for schema-based validation.

- **Email:** Standard regex format check.
- **OTP:** Exactly 6 numeric chars.
- **CSV:** Header validation + non-negative data check.

VII DEVELOPER IMPLEMENTATION GUIDE

26. Initial SQL Migration (Full Blueprint)

```
-- 1. Create Role Enum
CREATE TYPE user_role AS ENUM ('supplier', 'retailer');

-- 2. Create Profiles
CREATE TABLE profiles (
  id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,
  role user_role NOT NULL,
  full_name TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Create Products
CREATE TABLE products (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  supplier_id UUID REFERENCES profiles(id),
  sku TEXT NOT NULL,
  name TEXT NOT NULL,
  stock_qty INT DEFAULT 0,
  price DECIMAL(10,2),
  UNIQUE(supplier_id, sku)
);

-- 4. Security
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Suppliers manage their items"
ON products FOR ALL USING (auth.uid() = supplier_id);
```

27. Data Input Specification

sku	product_name	base_price	current_stoc
APPL-001	Red Apple	1.50	500

28. Technical Boilerplates

PYTHON ML ROUTE

```
@app.post("/ml/forecast")
async def get_forecast(data: list):
    df = pd.DataFrame(data)
    m = Prophet().fit(df)
    future = m.make_future_dataframe(periods=7)
    forecast = m.predict(future)
    return forecast[['ds', 'yhat']].to_dict('records')
```

29. Developer's "Day 1" Action Checklist

1. **Initialization:** `npx create-vite`, install `@supabase/supabase-js`, `zustand`, `lucide`, `react-router`, `axios`, `papaparse`.

2. **Auth Setup:** Enable Email OTP in Supabase Console, Link project key to `.env.local`.

3. **Execution:** Apply SQL Migration (Section 26) and build the Dual-Role entry card UI.

VIII PROJECT MANAGEMENT & VERIFICATION

30. 12-Week Development Roadmap

Weeks 1-2: Setup & Schema

Initialize Dev Environment & DB Definitions.

Weeks 3-4: Identity & Auth

Build OTP UI & Role Redirection logic.

Weeks 5-6: Supplier Tools

Catalog Dashboard & CSV Bulk Upload engine.

Weeks 7-8: Marketplace Sync

Real-time browse & stock updates via WebSockets.

Weeks 9-10: AI Intelligence

FastAPI microservice & B2C Forecasting UI.

Weeks 11-12: Launch & Polish

Testing, UI refinement & Production Deploy.

31. Risk Assessment & Mitigations

Risk	Impact	Mitigation Strategy
Backend Cold Start	High Latency	"Waking up" UI loading states with progress bars.
Data Privacy	High	Strict Supabase RLS per table per tenant.
Free Tier Limits	Medium	Batch querying and client-side caching.

32. Testing Strategy

Quality assurance utilizing standard free-tier testing libraries.

Unit (Vitest)
Store logic & ML utilities.

Integration (Supabase)
RLS verification & Auth flow.

E2E (Playwright)
Simulate Order-to-Sync flow.

33. Verification Plan (Final)

- **Auth Check:** Verify role restrictions (Retailer vs Supplier).
- **Real-time Check:** Instant stock sync across multiple browser sessions.

- **ML Accuracy:** Logical trend matching against mock data inputs.

IX CLOSING

34. Conclusion & Readiness Statement

The planning phase is now **100% complete**. We have established a blueprint that costs $0, scales to 50k users, uses Real-time WebSockets, and integrates Meta's Prophet for enterprise-grade forecasting.

35. Open Questions for Final Review

1. **OTP Method:** Confirm Email OTP is acceptable over Phone OTP (to stay 100% free).
2. **Hosting Delay:** Confirm 30s cold-start on Render Free tier is acceptable for the demo phase.

Project Identifier: 958d42cd-d403-49ba-af1d-b767cedc6b68